

Reviewer Pack — Plethysm Multiplicity Exponential Gap in Monotone Permanent ...

Entry 807f8a158fa7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 23:33:31 UTC

Plethysm Multiplicity Exponential Gap in Monotone Permanent vs Determinant

Entry ID: 807f8a158fa7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 23:33:31 UTC

1. Conjecture

Field A (mathematical branch): Plethysm Theory **Field B** (complexity object): Monotone Circuit Size

Statement:

For all $n \geq 2$, the multiplicity of the irreducible representation $\lambda = (n-1, 1)$ in the plethysm $\text{Sym}^{n(\text{Sym}^2(V))}$ decomposed for $V = \mathbb{C}^n$ is $\Omega(2^n)$, while for the determinant polynomial $\det_n$, this multiplicity is $O(1)$.

Rationale (proposer's reasoning):

Plethysm multiplicities encode symmetry constraints of polynomials. The exponential gap in $\lambda=(n-1,1)$ multiplicities between permanent (which requires symmetric power decompositions) and determinant (which factors via exterior powers) may reflect inherent structural complexity barriers for monotone circuits.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c1b5518c96f32a00

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def schur_functor(n, lambda_):
        if len(lambda_) != 2 or lambda_[0] != n - 1 or lambda_[1] != 1:
            return 0
        partition = (n-1, 1)
        sign = (-1)**sum(partition[i] * (partition[i] + 1) // 2 for i in range(len(partition)))
        product = math.factorial(n)
        hook_length_formula_value = hook_length_formula(partition, n)
        return sign * product // hook_length_formula_value

    def hook_length_formula(shape, n):
        result = 1
        for row in range(len(shape)):
            for col in range(len(shape[row])):
                hook_length = (n - row) + (n - col - shape[row][col]) - 1
                result *= hook_length
        return result

    def plethysm_multiplicity(n, lambda_):
        multiplicity = 0
        for partition in partitions(n):
            multiplicity += schur_functor(n, partition)
        return multiplicity

    def partitions(n):
        if n == 0:
            yield ()
            return
        for p in partitions(n - 1):
            yield (p + (1,))
            if len(p) > 0 and p[-1] > 1:
                yield tuple(x - 1 if x == p[-1] else x for x in p) + (1,)

    n = random.randint(2, 40)
```

```

plethysm_multiplicity = plethysm_multiplicity(n, (n-1, 1))

return {
    "metric_name": "plethysm_multiplicity",
    "metric_value": plethysm_multiplicity,
    "instances_tested": 1,
    "conjecture_holds": plethysm_multiplicity >= 2**n,
    "counterexample": "" if plethysm_multiplicity >= 2**n else f"plethysm_multiplicity < {2**n}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"plethysm_multiplicity < {2**n}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE conjecture_mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_421796fb.py", line 69, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_421796fb.py", line 54, in run_trial
    plethysm_multiplicity = plethysm_multiplicity(n, (n-1, 1))
                             ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_421796fb.py", line 40, in plethysm_mult
    for partition in partitions(n):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_421796fb.py", line 50, in partitions
    if len(p) > 0 and p[-1] > 1:
                          ~~~~~^
TypeError: '>' not supported between instances of 'tuple' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in partition comparison, preventing data collection | next:
Fix the partitions function to handle tuple comparisons properly

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	88117
2	propose	ollama_remote	qwen3:8b	0	0	74601
3	preregistration	ollama_remote	qwen3:8b	0	0	24065
4	novelty	ollama_remote	qwen3:8b	0	0	20795
5	novelty	ollama_remote	qwen3:8b	0	0	16795
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12669
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8541
8	judge	ollama_remote	qwen3:8b	0	0	17570

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 263153 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/807f8a158fa7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/807f8a158fa7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/807f8a158fa7.tar.gz` (if generated)